

URBAN FORK

Restaurant App MVP

Software Requirements Specification (SRS)

Frontend Only - React Native - Fall 2026

Assignment	Assignment No. 1
Student Name	
Registration No.	
Repository	restaurant-app-mvp
Brand	Urban Fork

Prepared for academic submission

Question 1: Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This Software Requirements Specification defines version 1.0 of Urban Fork, a frontend-only React Native restaurant application. It describes the customer and restaurant-manager experiences, local mock data, navigation, validation, state handling, usability goals, and the MVP screens to be implemented. The intended audience is the course instructor, student developer, and anyone reviewing or testing the prototype.

1.2 Scope

Urban Fork helps customers discover meals quickly, reserve a table, review prices before ordering, place a dine-in or takeaway order, and follow order progress. Restaurant managers can handle incoming orders, reservations, and menu availability from a dedicated dashboard. The application is entirely client-side.

- Customer login and signup using local mock users and role-based navigation.
- Menu browsing with Starters, Mains, Desserts and Drinks, plus specials, availability, search, sorting and favourites.
- Cart management with quantities, notes, local promo codes, item-count badge and live totals.
- Table reservation using mock tables and local reservation data.
- Dine-in and takeaway order selection with order summary and demo status tracking.
- Manager dashboard for orders, reservations and menu management.
- Light and dark theme support.
- Local persistence for orders, reservations and menu edits using AsyncStorage.

Out of scope:

- Real online payments or payment-gateway integration.
- Push notifications.
- Backend server, database, cloud authentication or external API.
- Real-time multi-device synchronization.
- Production deployment, encryption and payment security.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification; defines what the system must do.
MVP	Minimum Viable Product; the smallest working version demonstrating core features.
UML	Unified Modeling Language; diagrams for actors, classes, flows, states and components.
Hook	A React function such as useState or useEffect that adds React capabilities to functional components.
Context	React Context API used to share data without passing props through every intermediate component.

Reducer	A pure function that receives state and an action and returns a new state.
Mock Data	Local sample data used instead of a backend database or API.
FlatList	React Native component optimized for scrollable lists.
AsyncStorage	Local key-value storage used to persist selected application data.
Debounce	A delay technique used to wait until typing pauses before applying search.

2. Overall Description

2.1 User Roles

Role	Key Permissions & Goals
Customer / Diner	Create an account, log in, browse/search menu, manage favourites, add items to cart, apply promo codes, reserve a table, place an order, track status, view profile, switch theme and logout.
Restaurant Manager	Log in, access a manager-only dashboard, update incoming orders, accept/decline reservations, add menu items, edit prices, toggle availability, view profile, switch theme and logout.

2.2 Customer Self-Interview

As a potential Urban Fork customer, I would download the app if it helped me decide what to eat before reaching the restaurant, showed accurate prices and availability, let me reserve a table, gave me a clear bill before placing an order, and showed whether my food was pending, preparing, ready or served. I would also value favourites, recent searches, dark mode and the ability to add notes such as “less spicy” or “no onions.” These needs directly shape the user stories and functional requirements below.

2.3 User Stories

1. As a Customer, I want to browse the full menu with prices and availability, so that I can choose food before arriving.
2. As a Customer, I want to search for dishes by name or description, so that I can find food quickly.
3. As a Customer, I want to filter the menu by category, so that I can focus on relevant dishes.
4. As a Customer, I want to save favourite dishes, so that I can quickly return to items I like.
5. As a Customer, I want to reserve a table for a selected date and time, so that I can avoid waiting.
6. As a Customer, I want to add special instructions to cart items, so that my meal matches my preference.
7. As a Customer, I want to apply a valid promo code, so that I can see the discount before ordering.
8. As a Customer, I want to see a detailed order summary, so that I understand charges, tax, discount and total.
9. As a Customer, I want to choose Dine-in or Takeaway, so that the order matches how I plan to receive it.
10. As a Customer, I want to track order progress, so that I know when the food is ready.
11. As a Restaurant Manager, I want to view and update incoming orders, so that kitchen progress is reflected in the app.
12. As a Restaurant Manager, I want to accept or decline reservations, so that table usage remains manageable.

13. As a Restaurant Manager, I want to add menu items, edit prices and toggle availability, so that the customer menu stays current.
14. As a Restaurant Manager, I want to use a role-restricted dashboard, so that customer-only screens do not appear in my workflow.

3. Functional Requirements

Module	ID	Requirement
Authentication	FR-01	The system shall allow switching between Login and Signup modes on one screen.
Authentication	FR-02	The system shall validate email format, password length of at least eight characters, at least one digit, and matching confirmation password.
Authentication	FR-03	The system shall authenticate login credentials against local mock users after a simulated one-second delay.
Authentication	FR-04	The system shall route Customer users to customer navigation and Manager users to the manager dashboard.
Authentication	FR-05	The system shall store the authenticated user in AuthContext and support logout.
Menu	FR-06	The system shall load at least fifteen menu items across Starters, Mains, Desserts and Drinks from local mock data.
Menu	FR-07	The system shall display name, description, price, image, category, availability and Daily Special state for each menu item.
Menu	FR-08	The system shall grey out unavailable items and disable their Add button.
Menu	FR-09	The system shall support category filtering and sorting by price low-to-high, price high-to-low and name A-to-Z.
Menu	FR-10	The system shall allow the customer to toggle favourite menu items.

Search	FR-11	The system shall provide a search input that filters menu items by entered text.
Search	FR-12	The system shall debounce search input by 400 milliseconds.
Search	FR-13	The system shall retain the last five non-duplicate search terms as tappable suggestions.
Search	FR-14	The system shall show a Back to Top control after the menu has been scrolled more than 300 pixels.
Search	FR-15	The system shall display a friendly empty state when no menu item matches the query.
Cart	FR-16	The system shall allow available menu items to be added to the cart.
Cart	FR-17	The system shall allow quantities to be incremented and decremented and remove an item when quantity reaches zero.
Cart	FR-18	The system shall allow special instructions to be stored per cart item.
Cart	FR-19	The system shall apply valid local promo codes and reject invalid codes with a clear message.
Cart	FR-20	The system shall display a live cart-item-count badge.
Reservation	FR-21	The system shall offer hourly reservation slots from 12:00 to 22:00.
Reservation	FR-22	The system shall disable a time slot when no table with enough seats is available.
Reservation	FR-23	The system shall validate date, party size 1-12, Pakistani phone format 03XX-XXXXXXX, and one-hour booking lead time.
Reservation	FR-24	The system shall show a booking summary before creating a reservation and allow cancellation after confirmation.
Orders	FR-25	The system shall allow the customer to choose Dine-in or

		Takeaway before placing an order.
Orders	FR-26	The system shall calculate subtotal, 5 percent service charge, 15 percent sales tax, promo discount and grand total.
Orders	FR-27	The system shall create a new order with id, items, total, type, Pending status and timestamp.
Orders	FR-28	The system shall demonstrate Pending to Preparing to Ready to Served progression using demo timers.
Orders	FR-29	The system shall show a progress indicator and elapsed time on the Order Tracking screen.
Manager Dashboard	FR-30	The system shall display the Manager Dashboard only when the authenticated role is manager.
Manager Dashboard	FR-31	The system shall allow managers to update incoming order status manually.
Manager Dashboard	FR-32	The system shall allow managers to accept or decline reservations.
Manager Dashboard	FR-33	The system shall allow managers to add menu items, edit prices and toggle availability.
Manager Dashboard	FR-34	The system shall make manager menu changes immediately visible to customer screens through shared state.
Manager Dashboard	FR-35	The system shall persist orders, reservations and menu edits in AsyncStorage and restore them at app start.

4. Non-Functional Requirements

Area	Requirement
Usability	Controls shall use readable labels, validation feedback, loading indicators, disabled states and confirmations. Core customer tasks should be reachable in a small number of taps.
Performance	FlatList shall be used for menu rendering; simulated loading shall use the required delays; filtering, sorting and totals shall use memoization where appropriate.

Responsiveness	Layouts shall adapt to common phone sizes, use Safe Area handling and avoid clipped content in portrait orientation.
Maintainability	Source code shall be separated into components, screens, context, reducers, hooks, data and navigation with reusable UI and logic.
Data Handling	No backend shall be required. Initial data shall be stored in local JavaScript/JSON and working state shall use React state, Context, reducers and AsyncStorage where required.
Reliability	Timers and intervals shall be cleared during effect cleanup. Invalid forms shall not create orders or reservations.
Accessibility	Buttons and fields shall use descriptive text and sufficient touch targets; colour alone shall not be the only status indicator.
Compatibility	The project shall run as a React Native Expo application and be demonstrable using Expo Go.

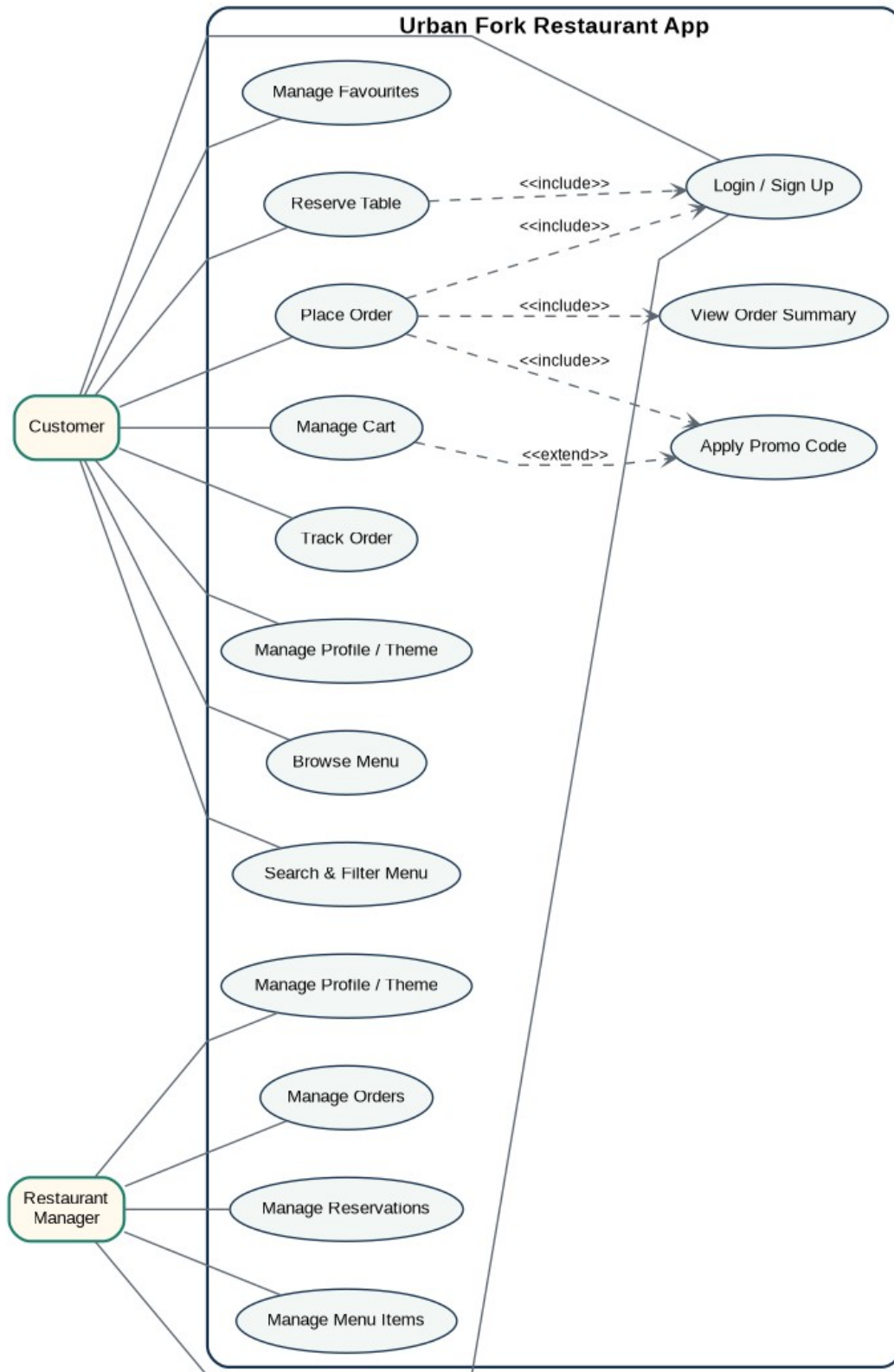
5. Client-Side Data Model (Mock Data)

Data Set	Fields	Description
Users	id, fullName, email, password, role	Mock customer and manager accounts for authentication and role navigation.
Categories	id, name	Menu grouping labels such as Starters, Mains, Desserts and Drinks.
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	All menu items shown to customers and editable by managers.
Tables	id, number, seats, isAvailable	Restaurant tables used for reservation availability.
Reservations	id, customerId, date, time, partySize, tableId, phone, status	Bookings created by customers and accepted, declined or cancelled.
Orders	id, customerId, items, total, type, tableId/pickupTime, status, timestamp	Placed orders and their local tracking state.
PromoCodes	code, discountPercent	Local valid promo codes such as FORK10 and GREEN20.
Favourites	customerId, menuItemIds	Local list of customer favourite menu item ids.
SearchHistory	customerId, terms	Up to five recent non-duplicate

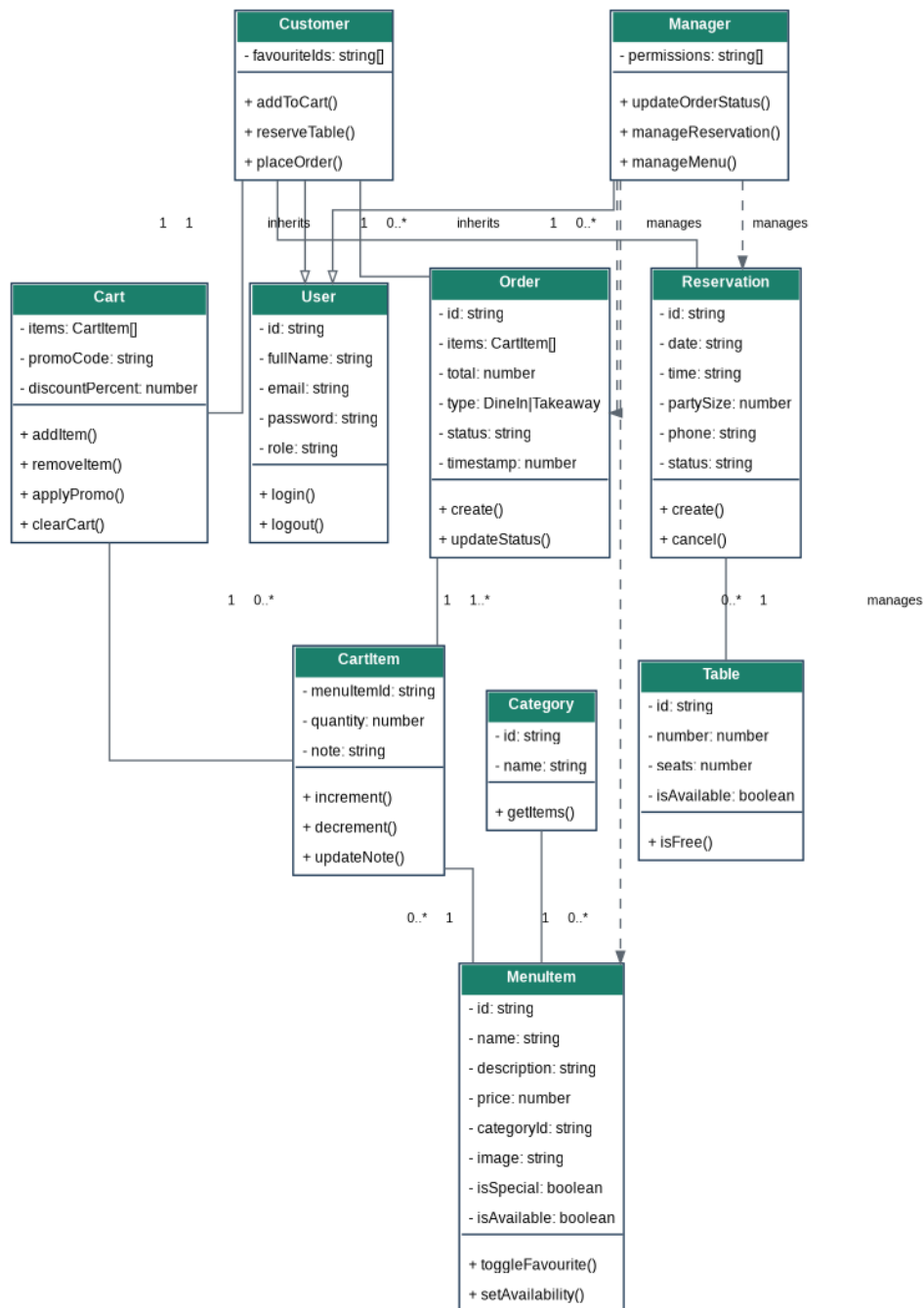
		search terms.
--	--	---------------

6. UML Diagrams

Figure 1. Use Case Diagram

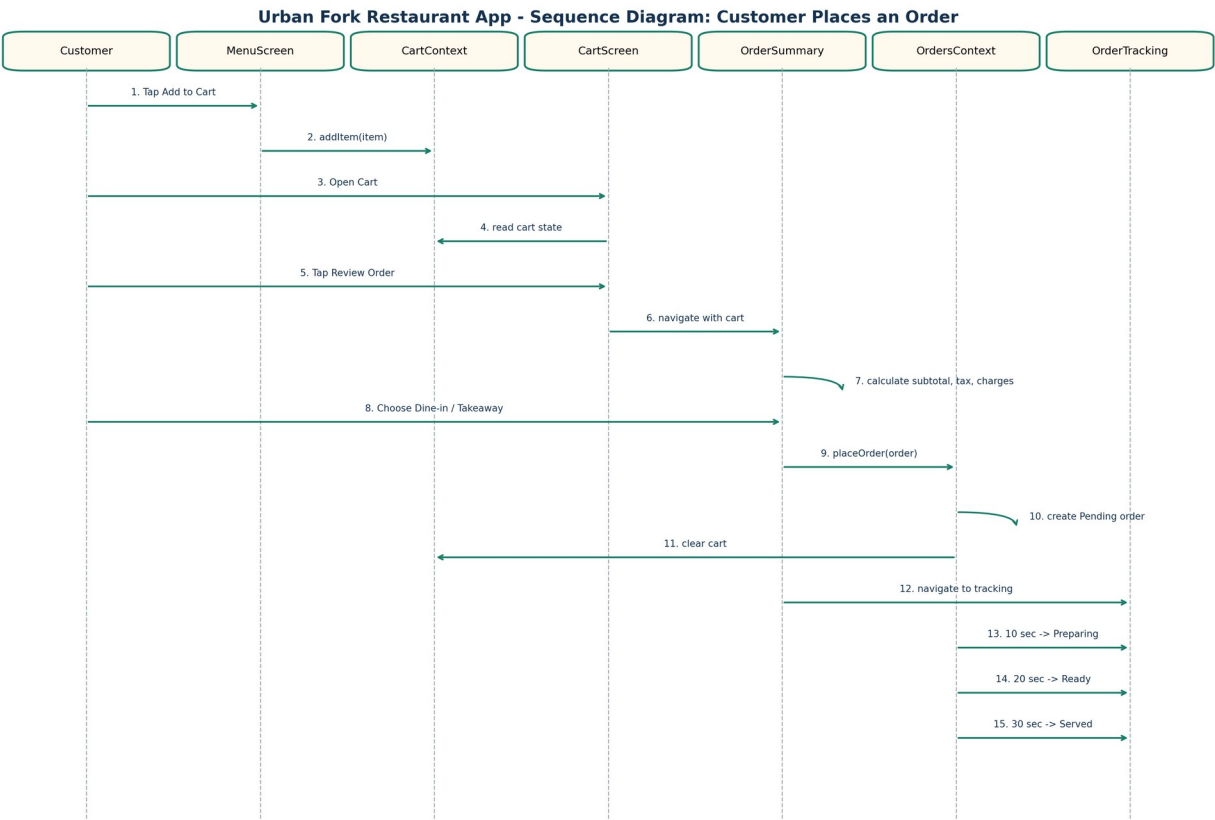


Shows the Customer and Restaurant Manager actors and their main interactions with Urban Fork. Include and extend relationships show dependencies around authentication, order review and promo-code use.

Figure 2. Class Diagram

Models User, Customer, Manager, Category, MenuItem, Cart, CartItem, Order, Reservation and Table, including inheritance, associations and multiplicities.

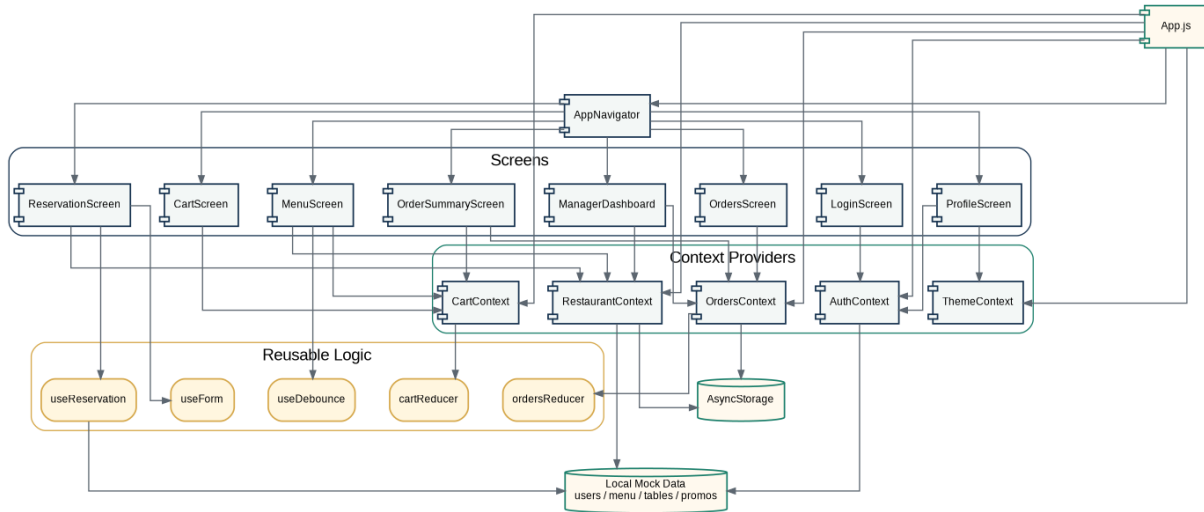
Figure 3. Sequence Diagram - Customer Places an Order



Shows the end-to-end order flow from Add to Cart through cart review, order summary, order creation and the timed Order Tracking states.

Figure 4. State Machine Diagram - Order Lifecycle

Shows Pending, Preparing, Ready, Served and Cancelled states with timer and manager events that trigger transitions.

Figure 5. Component Diagram

Shows App.js, navigation, screens, Context providers, reducers, custom hooks, mock data and AsyncStorage dependencies in the React Native frontend.

7. MVP Frontend Development (React Native)

7.1 Login and Signup Screen

Purpose	Allow users to authenticate or create a local account and select Customer or Manager role.
UI Elements	Login/Signup toggle, full name, email, password, confirm password, role selector, show-password control, validation messages, ActivityIndicator and submit button.
Navigation Entry / Exit	Entry: app start or logout. Exit: Customer Menu or Manager Dashboard.
Local Data Used	Users mock array.
Hooks and Why	useState for UI/form state; later useForm and AuthContext for reusable validation and global auth.

7.2 Menu Browsing Screen

Purpose	Display Urban Fork dishes, category filters, specials, availability, favourites and add-to-cart actions.
UI Elements	Category chips, FlatList cards, Daily Special badge, favourite heart, Add button, pull-to-refresh, loading/error/Retry UI.
Navigation Entry / Exit	Entry: customer login or Menu tab. Exit: Cart, Reservation, Profile and item actions.
Local Data Used	Categories and MenuItems.

Hooks and Why	useState for loading/error UI; useEffect for simulated fetch/header/cleanup; useMemo for derived filter/search/sort; useCallback for stable handlers.
---------------	---

7.3 Search and Scroll Controls

Purpose	Help customers quickly locate dishes and move around a long menu.
UI Elements	Search TextInput, search icon, clear button, recent searches, render counter, Back to Top, empty state.
Navigation Entry / Exit	Entry: part of Menu screen. Exit: selecting a result or clearing search.
Local Data Used	MenuItems and SearchHistory.
Hooks and Why	useRef for TextInput, FlatList, render counter and previous query; useDebounce for delayed search; useState for visible query UI.

7.4 Profile and Theme Screen

Purpose	Show logged-in user information and allow global theme switching and logout.
UI Elements	Name, email, role, light/dark switch and logout button.
Navigation Entry / Exit	Entry: Profile tab. Exit: other tabs or Login after logout.
Local Data Used	Authenticated User and theme palettes.
Hooks and Why	useContext through useAuth and useTheme to access shared auth/theme values.

7.5 Cart Screen

Purpose	Manage selected dishes and instructions before checkout.
UI Elements	Cart list, quantity stepper, remove button, note field, promo input, totals preview and Review Order button.
Navigation Entry / Exit	Entry: Cart tab or Add-to-Cart flow. Exit: Menu or Order Summary.
Local Data Used	Cart items and PromoCodes.
Hooks and Why	useReducer through CartContext for related cart transitions; useContext through useCart.

7.6 Order Summary Screen

Purpose	Show final price calculation and delivery type before
---------	---

	an order is placed.
UI Elements	Item summary, subtotal, service charge, tax, discount, grand total, Dine-in/Takeaway selector, table/pickup time and Place Order button.
Navigation Entry / Exit	Entry: Cart. Exit: Order Tracking or back to Cart.
Local Data Used	Cart state and selected order type/table/pickup time.
Hooks and Why	useMemo for subtotal, charges, discount and grand total; useCallback where handlers are passed to memoized children.

7.7 Table Reservation Screen

Purpose	Create and cancel local reservations with availability checking.
UI Elements	Date, time slots, party size, phone, table result, confirmation modal, My Reservations and cancel control.
Navigation Entry / Exit	Entry: Reservation tab. Exit: tab navigation or after booking confirmation.
Local Data Used	Tables and Reservations.
Hooks and Why	useReservation contains business logic; useForm handles reusable form state/validation; useDebounce is reused where needed.

7.8 Order Tracking and Manager Dashboard

Purpose	Track customer orders and provide manager controls over orders, reservations and menu items.
UI Elements	Tracking progress/elapsed timer; manager sections for Incoming Orders, Reservations and Menu Management.
Navigation Entry / Exit	Entry: after Place Order for customer; manager login for dashboard. Exit: tabs/profile/logout.
Local Data Used	Orders, Reservations and MenuItem persisted in AsyncStorage.
Hooks and Why	useEffect for timers, cleanup and storage; OrdersContext with useReducer integrates global order state.

Appendix A: Required Implementation Notes

A.1 Filtering dependency note (Q4)

If a filtering effect has an empty dependency array, it runs only after the first mount. Changes to the selected category or loaded menu items would not trigger the effect again, so the displayed list could become stale. When

state is used for derived filtering, the changing inputs must be dependencies. In the optimized version, derived filter/search/sort output should be calculated with useMemo instead of stored as separate state.

A.2 Context vs prop drilling (Q6)

Authentication and theme data are required by screens at different levels of the navigation tree. Context lets screens read the current user or theme directly without passing the same props through intermediate components that do not need them. It gives login, logout and theme switching one shared source of truth. Focused hooks such as useAuth and useTheme keep screen code clear. Role-based navigation can react to AuthContext immediately. A drawback is that consumers may re-render when the provided context value changes, so context values should be designed carefully.

A.3 useReducer vs useState for Cart (Q7)

The cart has many related transitions: add, remove, increment, decrement, update note, clear cart, apply promo and remove promo. useReducer centralizes these transitions in one pure function and makes changes predictable through explicit actions. This structure is easier to test and maintain when several fields change together. useState is still appropriate for simple isolated values such as a search query, selected category or modal visibility.

A.4 Reducer Test Cases (Q7)

Action	Initial State	Expected State
ADD_ITEM	items=[]	Selected item exists with quantity=1.
INCREMENT	item quantity=1	Same item quantity=2.
DECREMENT	item quantity=1	Item is removed when quantity reaches zero.
UPDATE_NOTE	note=""	Item note equals the entered instruction.
APPLY_PROMO	promoCode="", discountPercent=0	Valid promo stored and discount updated.
REMOVE_PROMO	discountPercent>0	Promo code and discount reset.
CLEAR_CART	items contain products	items=[] and promo fields reset.

A.5 When not to use useMemo/useCallback (Q8)

useMemo and useCallback should not be added to every value or function automatically. They have bookkeeping cost and can make code harder to read. They are most useful when a calculation is expensive, a derived list is recomputed often, or stable references prevent unnecessary rendering of memoized children. For small calculations and ordinary event handlers without a measurable performance problem, normal values and functions are simpler. Optimization should be used for a clear reason rather than by default.